

연락처 관리 웹 서비스 — 화면 정의서 (2차 과제)

항목	내용
문서명	연락처 관리 웹 서비스(2차 과제) 화면 정의서
문서 유형	Screen Definition (UI Specification)
과제 구분	2차 과제 — FastAPI + DB 연락처 프로그램
선행 과제	1차 과제 — 콘솔 연락처 관리 프로그램
버전	v1.0
환경	웹 브라우저 (HTML + JavaScript) / 개발자용 Swagger UI
상태	확정(Baseline)

이 문서의 역할(안내) 사용자가 마주하는 모든 화면을 하나씩 정의합니다. 각 화면이 무엇을 보여주고, 어떤 입력을 받고, 어떤 API를 호출해서 다음 어디로 이동하는지를 규정합니다. 1차 과제에서 "화면"은 콘솔에 출력되는 텍스트 한 장면이었지만, 2차 과제의 화면은 **브라우저에 그려지는 HTML 영역**입니다. 그리고 2차 과제에는 화면이 한 종류 더 있습니다 — 개발자가 API를 테스트하는 **Swagger UI**([/docs](#)) 입니다.

1. 화면 구성 전략 — "한 페이지, 두 개의 섹션" (아키텍처 결정)

본 과제의 웹 화면은 **HTML 파일 1장(GET /)** 으로 만들고, 그 안의 두 섹션을 **보였다/숨겼다** 하는 방식으로 구성합니다.

```
index.html (한 장)
├── [섹션 A] 로그인 화면  ← 로그인 전에만 보임
└── [섹션 B] 관리 화면    ← 로그인 후에만 보임
```

왜 페이지를 여러 장 만들지 않나요? (설계 근거)

이유	설명
배운 것만으로 가능	섹션 전환은 JavaScript로 <code>style.display</code> 를 바꾸는 것 — 이미 배운 DOM 조작 그대로입니다
페이지 이동 개념 불필요	여러 장으로 나누면 "이동할 때마다 로그인 상태를 다시 확인"하는 로직이 페이지마다 반복됩니다
실무 연결	실무의 SPA(Single Page App — 예: Gmail, 카카오톡)가 정확히 이 방식의 확장판입니다. "페이지를 갈아끼우지 않고 화면 일부만 갱신"하는 감각을 여기서 처음 익힙니다

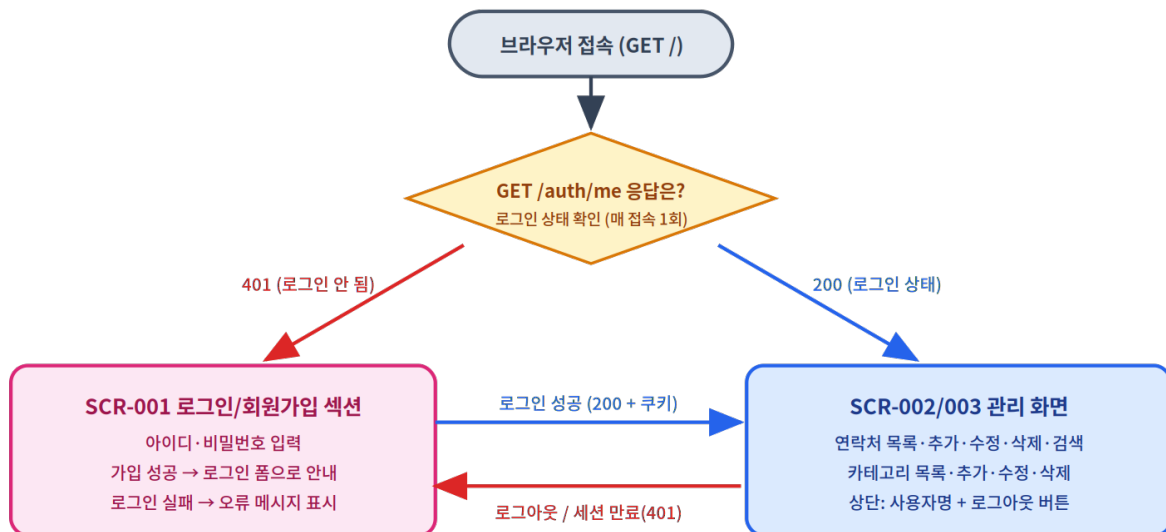
비유로 이해하기 — "연극 무대의 장면 전환" 극장(브라우저 탭)은 하나입니다. 조명이 바뀌면(로그인 성공) 무대 위 세트가 로그인 장면에서 관리 장면으로 바뀔 뿐, 관객이 극장을 옮기지 않습니다. 1차 과제의 콘솔도 사실 같은 구조였습니다 — 터미널 창(극장)은 하나이고 출력 내용(장면)만 바뀌었죠.

2. 화면 목록 (Screen Inventory)

화면 ID	화면명	보이는 조건	대응 FR
SCR-001	로그인 / 회원가입 섹션	로그인 전 (또는 세션 만료)	FR-01, FR-02
SCR-002	관리 화면 — 연락처 영역	로그인 후	FR-05 ~ FR-08
SCR-003	관리 화면 — 카테고리 영역	로그인 후 (SCR-002와 같은 화면 안)	FR-09 ~ FR-12
SCR-900	공통 메시지 / 오류 표시	모든 화면	NFR-01
DEV-001	Swagger UI (/docs)	개발자 전용 (브라우저 접속)	전체 API

3. 화면 전이도 (전체 워크플로우)

화면이 언제 바뀌는지의 전체 흐름입니다. 핵심 규칙은 하나 — "어느 섹션을 보여줄지는 **GET /auth/me** 의 응답이 결정한다" 입니다.



1차 과제의 "모든 기능은 처리 후 메인 메뉴로 복귀"가 → "모든 처리 후 관리 화면의 목록을 새로 고침"으로 바뀐 구조

초보자 포인트: 1차 과제의 메인 메뉴(SCR-001)가 하던 "허브" 역할을 2차에서는 **관리 화면**이 합니다. 다른 점은, 2차에는 그 허브에 들어가기 위한 **문(로그인 화면)** 이 하나 생겼다는 것뿐입니다.

4. 화면 상세 정의

SCR-001 · 로그인 / 회원가입 섹션

- 목적: 사용자를 확인하고 세션을 시작한다. 처음 온 사용자는 가입부터 한다.
- 레이아웃 (와이어프레임)

연락처 관리 서비스

아이디 [happyday]

비밀번호 [●●●●●●●●]

[로그인] [회원가입]

⚠ 아이디 또는 비밀번호가
올바르지 않습니다

← placeholder: "영문 소문자·숫자 4~20자"

← placeholder: "4~20자"

← SCR-900 메시지 영역 (평소엔 비어 있음)

• 구성 요소와 동작

요소	사용자 동작	호출 API	성공 시	실패 시
로그인 버튼	아이디·비밀번호 입력 후 클릭	POST /auth/login	관리 화면 섹션으로 전환 + 데이터 로딩	401 → 메시지 영역에 detail 표시
회원가입 버튼	같은 입력값으로 클릭	POST /auth/signup	"가입 완료! 로그인해 주세요" 안내	409 (중복)/ 422 (형식) → detail 표시

입력 예시 안내 규칙 (1차 과제 규칙 계승): 1차 과제는 프롬프트에 종류(ex.가족, 친구, 기타): 처럼 예시를 괄호로 보여줬습니다. 웹에서는 같은 역할을 입력창의 **placeholder 속성**이 합니다. 모든 입력창에 placeholder로 형식 예시를 제공해야 합니다.

SCR-002 · 관리 화면 — 연락처 영역 (아키텍처)

로그인 후 보이는 메인 화면입니다. 아래 와이어프레임이 관리 화면 전체의 구조(아키텍처)입니다.

연락처 관리 서비스

happyday 님 로그아웃

연락처 추가 (POST /contacts)

이름 (1~5자)

01012345678

주소

추가

종류 ▼ (드롭다운)

이름으로 검색 (예: 윤아)

검색 / 전체

연락처 목록 — 총 3건 (GET /contacts 의 total)

윤아 · 01012345678 · 서울시 · 친구

수정 삭제

은혁 · 01011112222 · 수원시 · 기타

수정 삭제

윤아 · 01023456789 · 부산광역시 · 가족

수정 삭제

※ 동명이인 "윤아" 2건 — 각 행이 자기 id를 갖고 있어 버튼만 누르면 정확한 대상 지정

카테고리 관리 (SCR-003)

가족

수정

삭제

친구

수정

삭제

기타

수정

삭제

새 카테고리 (1~10자)

추가

※ 카테고리가 바뀌면 왼쪽 추가 폼의 드롭다운도 갱신

▼ 화면 하단: SCR-900 공통 메시지 영역 (성공/오류 안내가 여기 표시됨)

• 구성 요소와 동작

요소	사용자 동작	호출 API	성공 시 화면 반응
추가 폼 + [추가]	4개 항목 입력 후 클릭	POST /contacts	폼 비우기 + 목록 새로 고침 (GET /contacts)
종류 드롭 다운	목록에서 선택	(없음 — GET /categories 결과로 미리 채워짐)	-
검색창 + [검색]	이름 입력 후 클릭	GET /contacts?name=윤아	목록을 검색 결과로 교체, "총 N건" 갱신
[전체]	클릭	GET /contacts	전체 목록으로 복귀
행의 [수정]	클릭	(호출 없음)	그 행의 값이 추가 폼에 채워지고 편집 모드로 전환 — [추가] 버튼이 [저장]으로 바뀜
편집 모드 [저장]	값 고친 후 클릭	PATCH /contacts/{id}	편집 모드 해제 + 목록 새로 고침
행의 [삭제]	클릭 → 확인 대화상자 (confirm)	DELETE /contacts/{id}	목록 새로 고침
[로그아웃]	클릭	POST /auth/logout	로그인 섹션(SCR-001)으로 전환

종류가 "입력"에서 "선택"으로 바뀐 이유 (중요한 설계 포인트): 1차 과제에서는 종류를 글자로 입력받아 `validate_type` 함수로 "가족/친구/기타 중 하나인지" 검사했습니다. 2차 과제에서는 **드롭다운(select)**으로 바뀌서,

애초에 목록에 있는 값만 고를 수 있게 합니다. 즉 "잘못된 입력을 검사"하는 대신 "잘못된 입력이 불가능한 UI"로 설계하는 것입니다. 실무 UI 설계의 기본 원칙이며, 1차 과제의 검증 함수 하나가 화면 설계로 대체되는 순간입니다. (물론 서버는 여전히 category_id를 검증합니다 — 화면을 거치지 않고 API를 직접 호출하는 경우가 있기 때문입니다. § 7 참조)

SCR-003 · 관리 화면 — 카테고리 영역

요소	사용자 동작	호출 API	성공 시	실패 시
새 카테고리 + [추가]	이름 입력 후 클릭	POST / categories	카테고리 목록 + 드롭다운 새로 고침	409 이름 중복 → detail 표시
항목의 [수정]	클릭 → 새 이름 입력(prompt)	PATCH / categories/{id}	목록 + 드롭다운 + 연락처 목록까지 새로 고침 (소속 연락처의 종류 표기가 바뀌므로)	409 이름 중복
항목의 [삭제]	클릭 → 확인 대화 상자	DELETE / categories/{id}	목록 + 드롭다운 새로 고침	409 사용 중 → detail ("...연락처가 N건 있어 삭제할 수 없습니다") 표시

SCR-900 · 공통 메시지 / 오류 표시 규칙

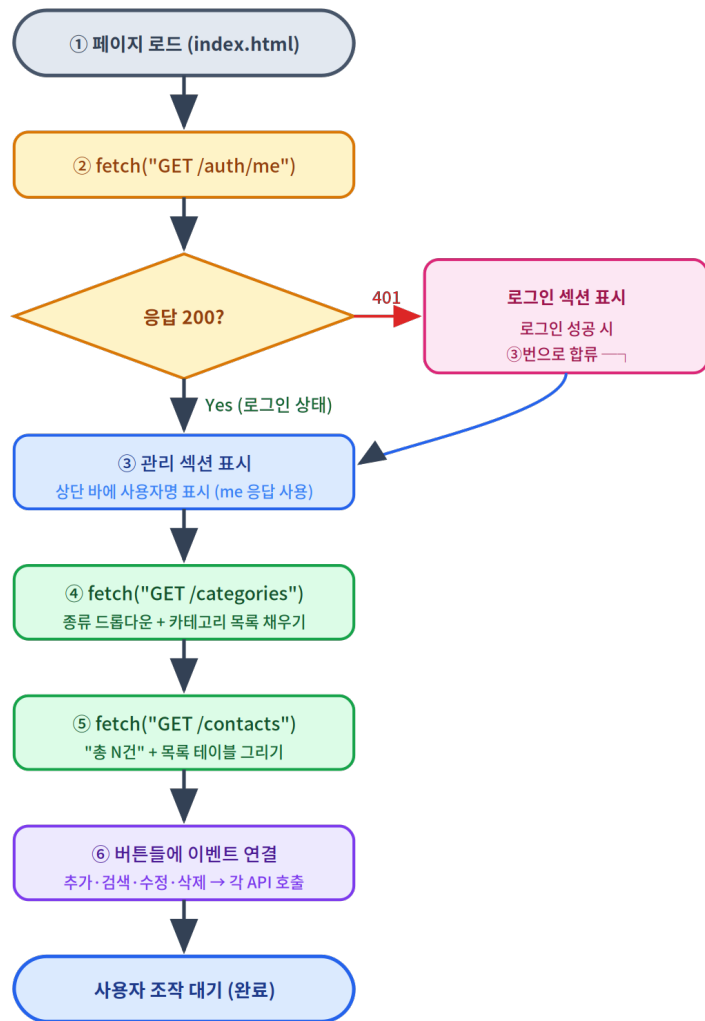
모든 API 실패는 {"detail": "..."} 형태로 오므로(01 문서 § 5-2), 화면의 오류 처리는 **한 가지 규칙**으로 통일합니다.

상황	화면 동작
응답이 401	(로그인 시도 자체의 401 제외) 세션 만료로 간주 → 로그인 섹션으로 전환 + "다시 로그인해 주세요"
응답이 404 / 409 / 422	응답의 detail 을 메시지 영역에 그대로 표시 (문구를 화면에서 새로 만들지 않음 — 서버가 단일 출처)
성공 (200/201/204)	초록색 안내 1~2초 표시 후 사라짐 (예: "추가되었습니다")
422의 detail이 목록 형태일 때	Pydantic 자동 422는 detail이 배열이므로, 첫 항목의 msg 만 뽑아 표시

1차 과제 대응: 1차의 SCR-900(공통 메시지)에 있던 "잘못된 입력입니다", "해당하는 회원 정보가 없습니다" 같은 문구들이, 2차에서는 **서버의 detail 문구**가 되어 내려오고 화면은 그것을 보여주지만 합니다. 문구의 주인이 화면에서 서버로 바뀐 것입니다 — 문구를 고칠 일이 생기면 서버 한 곳만 고치면 됩니다.

5. 단계별 워크플로우 — 페이지 로드부터 목록 표시까지

화면 코드(JavaScript)가 실제로 실행하는 순서입니다. "화면이 스스로 상태를 판단하고 데이터를 채우는" 이 흐름이 2차 과제 프론트엔드의 뼈대입니다.



④가 ⑤보다 먼저인 이유: 연락처 추가 폼의 **종류 드롭다운**은 카테고리 목록이 있어야 채울 수 있습니다. "화면의 어떤 부분이 어떤 데이터에 의존하는가"를 따져 호출 순서를 정하는 것 — 이것이 프론트엔드 설계의 기본 감각입니다.

6. 화면-기능-요구사항 매핑

화면 ID	호출 API	요구사항(FR)	1차 과제 대응 화면
SCR-001	POST /auth/signup, POST /auth/login	FR-01, FR-02	(없음 — 신규)
SCR-002	GET/POST /contacts, PATCH/DELETE /contacts/{id}	FR-05 ~ FR-08	SCR-001~005 (메뉴·추가·목록·수정·삭제)
SCR-003	GET/POST /categories, PATCH/DELETE /categories/{id}	FR-09 ~ FR-12	(없음 — 신규)
SCR-900	(모든 응답의 detail)	NFR-01	SCR-900 공통 메시지
DEV-001	전체 API	NFR-06	(없음 — 신규)

7. DEV-001 · 개발자 화면: Swagger UI (/docs)

FastAPI가 자동 생성하는 API 테스트 화면입니다. **HTML 화면을 만들기 전에, 모든 API를 여기서 먼저 검증하는 것이 이 과제의 표준 작업 순서입니다.**

사용 순서

1. 브라우저에서 `http://127.0.0.1:8000/docs` 접속
2. `POST /auth/signup` 펼치기 → **Try it out** → 본문 입력 → **Execute** → `201` 확인
3. `POST /auth/login` 실행 → `200` 확인 — 이때 브라우저에 세션 쿠키가 저장됨
4. 이후 `GET /contacts` 등 보호 API를 실행하면 **쿠키가 자동 첨부**되어 그대로 동작 (Swagger UI가 같은 주소에서 열리므로 브라우저가 쿠키를 함께 보냄)
5. `POST /auth/logout` 실행 후 보호 API 재실행 → `401` 확인 — 세션 삭제가 실제로 동작하는지 확인

화면 없이도 API를 검증할 수 있어야 하는 이유: 실무에서 백엔드와 프론트엔드는 보통 다른 사람이 만듭니다. 백엔드 개발자는 화면이 완성되기 전에 자기 API가 맞는지 스스로 증명해야 하고, 그 도구가 Swagger UI(또는 테스트 코드)입니다. 또한 §4의 드롭다운처럼 화면이 잘못된 입력을 막아 준다 해도, **API는 화면을 거치지 않고 직접 호출될 수 있으므로** 서버 검증은 절대 생략할 수 없습니다 — Swagger UI에서 일부러 잘못된 값을 넣어 보는 것이 그 확인 방법입니다.

8. 화면 공통 규칙

규칙	내용	1차 과제 대응
예시 안내	모든 입력창에 placeholder로 형식 예시 제공	프롬프트의 (ex: ...) 괄호 안내
처리 후 갱신	추가/수정/삭제 성공 시 관련 목록을 즉시 새로 고침	"처리 후 메인 메뉴 복귀"
파괴적 동작 확인	삭제는 반드시 확인 대화상자(confirm)를 거침	(신규 — 웹 표준 관례)
견고성	어떤 응답(401/404/409/422)에도 화면이 멈추지 않고 메시지 표시	"어떤 입력에도 프로그램이 죽지 않음"
한글 처리	<meta charset="UTF-8"> 명시	출력 인코딩 보장
이중 제출 방지	요청 진행 중에는 버튼 비활성화 (연타로 중복 등록 방지)	(신규 — 웹 특유의 문제)